3. Coding a transformer from scratch (30 points)

In this question you will fill in code to implement a decoder only, GPT-2 style transformer, and a simple training loop.

For part (a), we have included unit tests for each sub problem that can run locally on your laptop. You will be awarded full points for the subproblem if you pass the unit test. Do not edit the unit test file as we will separately be running the tests when you submit your code.

The following tips might be useful during this part of the assignment:

- Add assert statements to check the shape of tensors matches what you think it should be.
- Consider the Jaxtyping package to type hint the shape of tensors.
- Consider the einops package for manipulating tensors (`einops.rearrange` is particularly useful).

This will help you not only write less buggy code, but also make your code far more readable.

(a) (20 points) In this part of the question we will implement a transformer in the `model_solution.py` file. The file contains a number of different classes that you will implement. In the end, you will have an implementation of the `Transformer` class with functioning `forward` and `generate` methods. In part (b), we will (start to) train your implementation of `Transformer`.

i. (0 points) Familiarize yourself with the classes in the `model_solution.py` file. We will ask you to implement them in the order `MLP`, `CausalAttention`, `DecoderBlock`, and finally `Transformer`. We will get you to implement the classes in this order because it is the order of dependence. `Transformer` depends on `DecoderBlock`, that in turn depends on `CausalAttention` and `MLP`.

ii. (1 point) Implement `MLP.forward`. Check you pass the corresponding test.

iii. (6 points) Implement `CausalAttention.forward`. Check you pass the corresponding test.

iv. (2 points) Implement `DecoderBlock.forward`. Check you pass the corresponding test.

v. (6 points) Implement `Transformer.forward`. Check you pass the corresponding test.

vi. (5 points) Implement `Transformer.generate`. Check you pass the corresponding test. Note: you should implement greedy decoding for this function..

(b) (10 points) After finishing part (a), you now have a functioning Transformer model. If you look at `Transformer.__innit__` we can see that when you create an instance of the `Transformer` class, we initialize the model with random weights according to the `Transformer._init_weights` method. In this part of the question, you will implement a training loop, and start training a small model locally on your laptop.

i. (0 points) Look in the `train.py` file and familiarize yourself with the training loop. We will run this code to train the model.

ii. (7 points) First, implement `Transformer.get_loss_on_batch`. This function maps a batch of tokens to a single loss value. We use this function in `train.py` to get the loss over a batch. Check you pass the corresponding test.

iii. (3 points) Run `train.py`. This will train the model, using your `Transformer.get_loss_on_batch` on 100 batches of data. At the end of training it will save a graph of the training loss and gradient norm over training to `losses_and_grad_norms.png`, include an image of this below.
If everything is correct, you should see a decreasing loss curve.

(c) (9 points) **(Bonus)** In this optional bonus question, your goal is to speed up the learning process. We will keep the number of gradient steps fixed to 100, however you can change anything else about `train.py` or `model.py` to speed up training.

We will consider a change to have succeeded if the final loss after 100 steps is lower than the baseline curve you reported in part biii).

We will award 3 points for each different change you make that leads to a speedup. Thus for full points, you will need to make three different changes to the training file, each of which leads to a speedup. These changes should compound. For example, you may begin by changing the learning rate, leading to a lower loss. You then might keep this better learning rate, and combine it with a second change (e.g., changing the optimizer or model architecture), that leads to an even lower loss. Note that changing the learning rate to three different values counts as one idea: we are looking for three different types of ideas.

When you are done, submit:

- A description of each change that you made.
- Up to three new learning curves, one for each change you made; and additionally include the baseline from biii).
- The lowest loss you achieved after 100 steps.